City Tour

Fluffy the Squirrel is bored of staying at home and wants to go on a tour of the city when CMCO ends by jumping between buildings. The city contains M rows and N columns of buildings with varying heights, represented by a grid of numbers H of size $M \times N$, where the number in the i^{th} row and j^{th} column, $H(i, j)$ represents the height of the building at that position.

Fluffy's home is at $(1, 1)$. For safety reasons, Fluffy can only jump between buildings that are directly next to each other (not diagonally) and have a height difference strictly less than D. Formally, Fluffy can jump from (i, j) to (k, l) if and only if $|i - k| = 0, |j - l| = 1$ or $|i - k| = 1, |j - l| = 0$ is true, and $|H(i, j) - H(k, l)| < D$ is true.

Given the description of the city, can you help Fluffy find out how many buildings (including Fluffy's home) can be reached?

Constraints

- All of M, N, D and $H(i, j)$ are integers,
- $1 \leq M \leq 10^5$,
- $1 \leq N \leq 10^5$,
- $1 \leq D \leq 10^5$,
- $1 \leq M \times N \leq 10^5$, and
- $-10^6 \leq H(i, j) \leq 10^6$.

Sample Input

```
M = 4
N = 5
D = 5
H = [[1, 3, 7, 9, 16], [6, 2, 4, 1, 8], [8, 9, 10, 12, 14], [7, 5, 1, 4, 11]]
```

Sample Output

```
18
```

Explanation:

A diagram of the city is as shown:

```
[1,  3,  7,  9, 16]
[6,  2,  4,  1,  8]
[8,  9, 10, 12, 14]
[7,  5,  1,  4, 11]
```

The building in the first row and fifth column $(1, 5)$, with height $H(1, 5) = 16$ cannot be reached.
The building in the second row and fifth column $(2, 5)$, with height $H(2, 5) = 8$ cannot be reached.
It is possible for Fluffy to reach all other buildings by jumping between them.
Note that Fluffy can make jumps in *any* direction, so the building (2, 1) with height 6 can be reached by jumping from 1 -> 3 -> 2 -> 6 for example, although 1 -> 6 is prohibited.

Solution

One of the most common incorrect solutions for this problem is to only check if a building has a neighbour which Fluffy can jump to. Why is this wrong?

```
D = 2

[10, 10, 10, 10]
[10,  8,  8, 10]
[10,  8,  8, 10]
[10, 10, 10, 10]
```

In the case above, Fluffy can jump between any of the buildings in the middle, since they all have height 8.
However, Fluffy *cannot* jump into the middle in the first place, as doing so would require a jump from height 10 to height 8, which is not allowed when D = 2.
Fluffy could reach any of those four buildings if it started inside the middle, but its home is at (1, 1), so it actually cannot reach those buildings.

Now we know that we have to check if a building can be reached from Fluffy's home, not just from its neighbours.
One way to do so might be go through every building and check if there is a path from this building back to (1, 1), but this will be very slow.
Instead, it is better to start from (1, 1) and build a path outward to find which buildings can be reached.

The intuition for this process is that if you can reach a certain building A from (1, 1), and you can reach B from A, then you can reach B from (1, 1) by going through A.
Basically, we will build the path by starting with Fluffy's home at (1, 1), which is obviously able to be reached. Using the sample input as an example,

```
D = 5

[  1,  3,  7,  9, 16]
[  6,  2,  4,  1,  8]
[  8,  9, 10, 12, 14]
[  7,  5,  1,  4, 11]

[  R, - ,   ,   ,   ]
[ - ,   ,   ,   ,   ]
[   ,   ,   ,   ,   ]
[   ,   ,   ,   ,   ]

R represents a reachable building
- represents a neighbour of a reachable building
```

Then, for every reachable building, we check its neighbours.
If we can jump from an R to its neighbour, then the neighbour is also reachable, because Fluffy can jump from (1, 1) to R, then to the neighbour.
In the diagram above, 1 -> 3 is a valid jump but 1 -> 6 is not.
Therefore, the building with height 3 is R, but 6 is not (yet).

```
[  1,  3,  7,  9, 16]
[  6,  2,  4,  1,  8]
[  8,  9, 10, 12, 14]
[  7,  5,  1,  4, 11]

[  R,  R, - ,   ,   ]
[ - , - ,   ,   ,   ]
[   ,   ,   ,   ,   ]
```

```
[   ,   ,   ,   ,   ]
```

Notice that, at this stage, we do not yet mark (2, 1) as impossible to visit (in fact, we will not make any such marking in any stage).
This is because there is still the possibility that there can be a path to reach this building by jumping from somewhere else.
Out of the three neighbours, 3 -> 2 and 3 -> 7 are valid jumps, whereas 1 -> 6 is not.

```
[  1,  3,  7,  9, 16]
[  6,  2,  4,  1,  8]
[  8,  9, 10, 12, 14]
[  7,  5,  1,  4, 11]

[  R,  R,  R, - ,   ]
[ - ,  R, - ,   ,   ]
[   , - ,   ,   ,   ]
[   ,   ,   ,   ,   ]
```

Now, the building (2, 1) becomes reachable since the jump 2 -> 6 from the R at (2, 2) is indeed a valid jump.
As for the other jumps, 7 -> 9, 2 -> 4 and 7 -> 4 are valid, but 2 -> 9 is not.

```
[  1,  3,  7,  9, 16]
[  6,  2,  4,  1,  8]
[  8,  9, 10, 12, 14]
[  7,  5,  1,  4, 11]

[  R,  R,  R,  R, - ]
[  R,  R,  R, - ,   ]
[ - , - , - ,   ,   ]
[   ,   ,   ,   ,   ]
```

Repeating this process, we will eventually come to a point where every building has been checked, obtaining

```
[  1,  3,  7,  9, 16]
[  6,  2,  4,  1,  8]
[  8,  9, 10, 12, 14]
```

```
[  7,  5,  1,  4, 11]

[  R,  R,  R,  R, -]
[  R,  R,  R,  R, -]
[  R,  R,  R,  R,  R]
[  R,  R,  R,  R,  R]
```

The answer for the sample case is 18.

Now, you should have an intuition of how the process works.
We just need to implement the process in an efficient manner.

1. Define a boolean array `visited[]` that stores whether or not we have visited a building. This is initially false for all buildings.
2. Define a list/queue/stack/any other data structure `can_reach` that initially contains only Fluffy's home (1, 1).
3. For each building R in `can_reach`, perform the following actions:
    1. Change `visited[R] = true`, basically marking R as being reachable.
    2. For each neighbour of R, check if it has been visited already.
    3. If it has been visited, that means it is already reachable. Skip it.
    4. If it has not been visited, check if the height difference between it and R is less than D.
    5. If yes, then add it to `can_reach`. If not, do not do anything.
    6. Remove R from `can_reach`.
4. Perform step 3 on the elements of `can_reach` until it becomes empty. This does not mean step 3 is done only once, since step 3.v. will add new buildings to `can_reach` - it is just that we start from a single building at (1, 1).
5. The final values of `visited[]` will tell us which buildings can be visited.

In this process, `can_reach` is basically our set of reachable buildings R.
The checking of neighbours is done whenever a new building is added to R.
Only the neighbours that can be reached from this building are added to `can_reach`, and those will eventually all be added to R.
To prevent looking at a building again when it is already reached (since this wastes time), we use `visited[]` to store whether we can reach a building, and remove buildings from `can_reach` once we have processed their neighbours.

Implementation

```
visited = []
for x in range(M):
    tp_list = []
    for y in range(N):
        tp_list += [False]
    visited += [0]
    visited[x] = tp_list
# creates visited[] of size M*N and all elements are false

can_reach = [(0, 0)]
# start with (0, 0) instead of (1, 1) because of 0-indexing

while len(can_reach) > 0:
    # process the first element can_reach[0]
    # mark it as visited
    row = can_reach[0][0]
    col = can_reach[0][1]
    visited[row][col] = True

    # the neighbours are
    # (row-1, col), (row+1, col), (row, col-1), and (row, col+1)
    # we need to check that the neighbour is not visited
    # and its height difference is < D
    if ((row - 1 >= 0) and (!visited[row-1][col])
            and (abs(H[row-1][col] - H[row][col]) < D)):
        can_reach += [(row-1, col)]

    if ((row + 1 < M) and (!visited[row+1][col])
            and (abs(H[row+1][col] - H[row][col]) < D)):
        can_reach += [(row+1, col)]

    if ((col - 1 >= 0) and (!visited[row][col-1])
            and (abs(H[row][col-1] - H[row][col]) < D)):
        can_reach += [(row, col-1)]

    if ((col + 1 < N) and (!visited[row][col+1])
            and (abs(H[row][col+1] - H[row][col]) < D)):
        can_reach += [(row, col+1)]

    # remove the processed element from can_reach
    del can_reach[0]

# count and print how many buildings can be reached
cnt = 0
```

```
for x in range(M):
    for y in range(N):
        if visited[x][y]:
            cnt += 1
print(cnt)
```

Note

The problem of visiting all points in a graph is known as *graph traversal*, and the two main algorithms are depth-first search (DFS) and breadth-first search (BFS).
Both of these algorithms work very similarly to what we have described above, with the only difference being whether the valid neighbours are added to the beginning or the end of `can_reach`.

If they are added to the end (like we have done), then it is a *breadth-first search*.
Suppose we have added the neighbours of R to `can_reach`.
Then, the neighbours of the neighbours will be added to the end of `can_reach`, so the neighbours of R are guaranteed to be processed first, before the neighbours of neighbours.
This is a BFS because we finish processing the "broad" level of neighbours first before going into the "deep" levels (neighbours of neighbours).

On the other hand, if they are added to the start, then it is a *depth-first search* because the neighbours of neighbours will be added to the beginning.
This means the neighbours of neighbours will be processed before the other neighbours, so we are searching the "deep" levels before going to the "broad" level.